Universidad Nacional de Tucumán
Facultad de Ciencias Exactas y Tecnología
Computación Científica

Estimación de las constantes π y e mediante el Método de Monte Carlo

Implementación serial en C y paralela con MPI
Proyecto de aplicación de Computación de Alto Rendimiento

Integrantes

Ason, Belén

Cano Arce, Valentina

Caballero, Leonardo

Repositorio: github.com/belenason/pi-y-e-metodo-montecarlo

Octubre de 2026

Índice

1. Introducción **3**

2. Objetivos **3**

3. Fundamentos teóricos **4**

3.1. Método de Monte Carlo 4

3.2. Estimación de π 4

3.3. Estimación de e 5

3.4. Convergencia y error 5

3.5. Limitaciones del método y estrategias de resolución 6

3.6. Métricas de rendimiento paralelo 7

4. Implementación **7**

4.1. Versión serial 7

4.2. Estrategia de paralelización 7

4.3. Números aleatorios y comunicación 8

4.4. Medición del tiempo 9

5. Diseño experimental **9**

6. Resultados y análisis **10**

6.1. Precisión: convergencia y error 10

6.2. Tiempos de ejecución 11

6.3. Speedup y eficiencia en el clúster 12

6.4. Speedup en la PC y comparación entre sistemas 13

6.5. Limitaciones del estudio 14

7. Conclusiones **14**

8. Mejoras futuras **15**

9. Bibliografía **16**

1. Introducción

Este proyecto consiste en la estimación de las constantes matemáticas π y e mediante el Método de Monte Carlo, aplicando técnicas de Computación de Alto Rendimiento (HPC) en el lenguaje C con el estándar MPI, mediante la implementación Open MPI, para analizar y mejorar el rendimiento de los cálculos. El objetivo de este proyecto no se limita a obtener una aproximación de estas constantes, sino que también busca estudiar cómo se comporta el problema al aumentar el número de simulaciones y al distribuir el trabajo entre diferentes procesos.

El Método de Monte Carlo es una técnica numérica que utiliza la generación de muestras aleatorias y el análisis estadístico de sus resultados para obtener aproximaciones a problemas deterministas. Su principal característica es que, en lugar de buscar directamente una solución exacta mediante un procedimiento determinista, realiza una gran cantidad de simulaciones aleatorias y utiliza el comportamiento conjunto de estas muestras para aproximar el resultado buscado.

La elección de este método resulta especialmente adecuada para los objetivos del proyecto debido a que las simulaciones pueden realizarse de manera independiente entre sí. Esto permite distribuir una gran cantidad de simulaciones entre diferentes procesos y unidades de procesamiento, haciendo que el problema sea apropiado para estudiar técnicas de paralelización. Además, la cantidad de simulaciones puede modificarse para analizar cómo influye en la precisión de los resultados, la convergencia y el tiempo de ejecución.

Si bien existen otros métodos para calcular o aproximar π y e, como series matemáticas, métodos de integración numérica o algoritmos iterativos, la elección de Monte Carlo no se basa únicamente en su capacidad para aproximar las constantes, sino también en que permite estudiar de manera directa la distribución del trabajo y el comportamiento de una implementación paralela.

Para el desarrollo del proyecto se implementó primero una versión serial y luego una versión paralela del problema. Sobre ambas se realizó un estudio experimental variando el número de simulaciones N y la cantidad de procesos P en dos sistemas de cómputo distintos: una computadora personal y un nodo de un clúster. Cada configuración se ejecutó múltiples veces y se utilizó la mediana de los tiempos para reducir la influencia de variaciones ocasionales del sistema, lo que permite analizar la convergencia del error, el *speedup* y la eficiencia paralela.

Finalmente, se estudian y comparan los resultados obtenidos para determinar el comportamiento de la implementación, sus limitaciones y las posibles mejoras que podrían realizarse en futuros casos.

2. Objetivos

Objetivo general. Desarrollar y analizar una implementación paralela basada en el Método de Monte Carlo para estimar las constantes matemáticas π y e, estudiando la precisión de las aproximaciones y el comportamiento del programa en términos de rendimiento computacional.

Objetivos específicos:

- Investigar el funcionamiento del Método de Monte Carlo y su aplicación para la estimación de π y e.
- Diseñar e implementar una estrategia de paralelización adecuada para el problema, distribuyendo las simulaciones entre los procesos de manera equilibrada.
- Analizar cómo influye la cantidad de simulaciones en la precisión de las estimaciones y verificar experimentalmente la tasa teórica del error, $1/\sqrt{N}$.
- Medir los tiempos de ejecución de las versiones serial y paralela, y calcular el *speedup* y la eficiencia a partir de la mediana de múltiples ejecuciones.
- Analizar el comportamiento de la implementación en diferentes sistemas de cómputo.
- Identificar las limitaciones de la implementación y proponer posibles mejoras para futuros casos.

3. Fundamentos teóricos

3.1. Método de Monte Carlo

El Método de Monte Carlo es una técnica numérica que utiliza el muestreo aleatorio y métodos estadísticos para obtener aproximaciones a problemas deterministas. En lugar de calcular directamente una solución exacta, se plantea un experimento probabilístico cuyo valor esperado coincida con el resultado buscado, se realizan numerosas simulaciones aleatorias y se utiliza su promedio para aproximar la solución [1], [2]. Como el resultado surge del comportamiento estadístico de muchas muestras, la cantidad de simulaciones realizadas (N) tiene una influencia directa sobre la precisión, y distintas ejecuciones producen estimaciones ligeramente diferentes.

Otra característica fundamental para este proyecto es la independencia entre las simulaciones: cada prueba puede realizarse sin depender del resultado de las demás. Por lo tanto, el promedio de una única serie de N simulaciones es estadísticamente equivalente al promedio de varias series más cortas ejecutadas en simultáneo, lo que permite dividir el conjunto de simulaciones en partes, asignarlas a distintos procesos y, al final, combinar los resultados parciales sin alterar el estimador, siempre que cada proceso utilice secuencias de números pseudoaleatorios independientes entre sí. Esta estructura convierte a Monte Carlo en un problema trivialmente paralelo (*embarrassingly parallel*) y lo hace especialmente adecuado para estudiar técnicas de Computación de Alto Rendimiento.

3.2. Estimación de π

Para comprender cómo se puede estimar π mediante el Método de Monte Carlo, primero es necesario establecer una relación geométrica que permita obtener esta constante a partir de una proporción de áreas. Si se inscribe un círculo de radio 1 en un cuadrado de lado 2, la proporción entre sus áreas es π/4. Por simplicidad, se trabaja únicamente con el primer cuadrante: un cuarto de círculo de radio 1, centrado en el origen, dentro del cuadrado de vértices (0, 0), (1, 0), (0, 1) y (1, 1). Debido a la simetría de ambas figuras respecto de los ejes cartesianos, la proporción entre las áreas se mantiene intacta. Además, las coordenadas quedan directamente en el intervalo [0, 1) que entrega el generador de números aleatorios, por lo que no es necesario transformarlas. El cuadrado funciona como región de referencia de área conocida, y el cuarto de círculo relaciona la proporción de áreas con π:

$$A_{cuadrado} = 1 \cdot 1 = 1, \quad A_{cuarto} = \frac{\pi \cdot 1^2}{4} = \frac{\pi}{4} \quad \Rightarrow \quad \frac{A_{cuarto}}{A_{cuadrado}} = \frac{\pi}{4} \tag{1}$$

¿Por qué tirar puntos en un cuadrado permite calcular π?

La idea fundamental del método es que, si los puntos se distribuyen uniformemente dentro del cuadrado, cada punto tiene la misma probabilidad de caer en cualquier zona. Por lo tanto, la proporción de puntos que cae dentro de una determinada región se aproxima a la proporción del área que ocupa esa región en relación con el área total. Si de N puntos generados M quedan dentro del cuarto de círculo:

$$\frac{M}{N} \approx \frac{A_{cuarto}}{A_{cuadrado}} = \frac{\pi}{4} \quad \Rightarrow \quad \pi \approx 4 \cdot \frac{M}{N} \tag{2}$$

Es decir, no es el cuadrado por sí mismo el que permite calcular π: los puntos aleatorios permiten estimar la proporción entre ambas áreas sin tener que calcular directamente el área del cuarto de círculo.

Generación y clasificación de los puntos

En cada simulación se generan dos números aleatorios x e y, uniformes en [0, 1), que definen un punto ubicado dentro del cuadrado. El punto pertenece al cuarto de círculo si su distancia al origen, obtenida mediante el teorema de Pitágoras, no supera el radio: $\sqrt{(x^2 + y^2)} \leq 1$. Como ambos miembros son positivos, la condición equivale a la Ec. 3, que evita calcular una raíz cuadrada en cada simulación. Al aumentar la cantidad de simulaciones, la proporción observada tiende a aproximarse cada vez más a la proporción real entre las áreas.

$$x^2 + y^2 \leq 1 \tag{3}$$

3.3. Estimación de e

La idea del método consiste en generar números aleatorios uniformemente distribuidos entre 0 y 1 y acumularlos hasta que la suma sea mayor que 1. La cantidad de números necesarios para superar 1 constituye el resultado de una simulación y, repitiendo este procedimiento una gran cantidad de veces, el promedio de las cantidades obtenidas tiende al valor de e.

Procedimiento de una simulación

Cada simulación comienza con una suma $S = 0$ y le suma números aleatorios uniformes en [0, 1) hasta que $S > 1$; la cantidad K de números necesarios es el resultado de la simulación. Por ejemplo, si se generan 0,25; 0,18; 0,31 y 0,42, las sumas acumuladas son 0,25 → 0,43 → 0,74 → 1,16 > 1, por lo que $K = 4$. Otra simulación podría necesitar solamente 2 números, y otra 3, 5 o una cantidad diferente: el resultado de cada simulación es una variable aleatoria.

¿Por qué este procedimiento permite estimar e?

El valor esperado de K es exactamente e, una propiedad conocida de la suma de números aleatorios uniformes cuya demostración puede consultarse en [3]. Por lo tanto, si K_i es el resultado de la simulación i, la estimación es el promedio:

$$e \approx \hat{e}_N = \frac{1}{N} \sum_{i=1}^{N} K_i \tag{4}$$

Cada simulación individual produce solo un número entero; es el promedio de muchas simulaciones el que aproxima la constante. A diferencia de π, donde cada simulación consume siempre dos números aleatorios, aquí cada una consume una cantidad aleatoria (en promedio, e ≈ 2,72), por lo que su costo es variable, un aspecto relevante para el reparto del trabajo entre procesos (sección 4.2). La Figura 1 ilustra ambos experimentos.

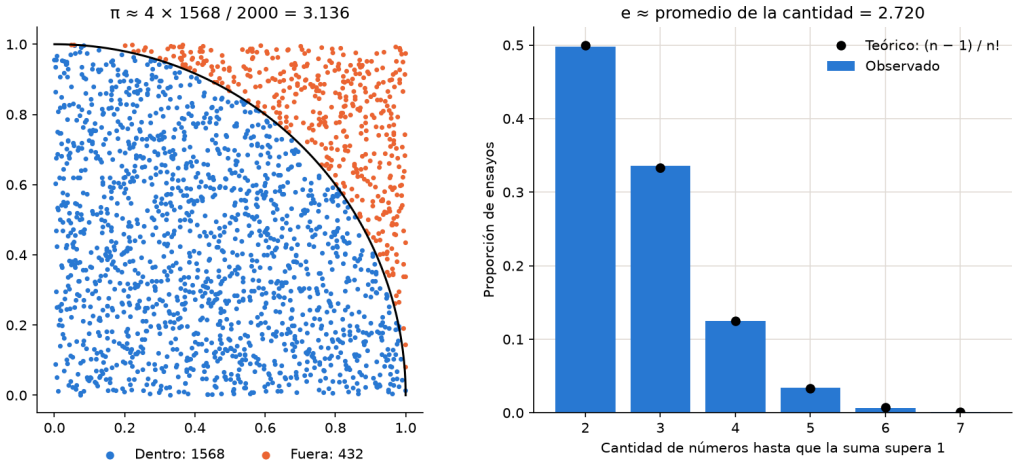


Figura 1. Ilustración de los dos experimentos con pocas muestras. Izquierda: 2.000 puntos aleatorios en el cuadrado unitario; los que caen dentro del cuarto de círculo (azul) permiten estimar π. Derecha: distribución de la cantidad de números uniformes necesarios para que su suma supere 1, frente a la probabilidad teórica $(n - 1)/n!$; su promedio estima e.

3.4. Convergencia y error

3.4.1. Convergencia

Ambas estimaciones se obtienen promediando variables aleatorias independientes e idénticamente distribuidas, ya que todas las simulaciones se generan bajo las mismas condiciones probabilísticas y sin influirse entre sí. Para el caso de π, cada punto generado constituye un ensayo con dos resultados posibles (caer dentro o fuera del cuarto de círculo), con probabilidad de acierto $p = \pi/4$, y el estimador es $\hat{\pi}_N = 4 \cdot M/N$. De manera análoga, e se aproxima mediante el promedio $\hat{e}_N$ de la Ec. 4. La Ley Fuerte de los Grandes Números garantiza que, a medida que el número de simulaciones tiende a infinito, ambos estimadores convergen a sus valores teóricos, π y e respectivamente [2].

3.4.2. Velocidad de convergencia

La Ley de los Grandes Números indica hacia dónde converge el método, pero no qué tan rápido lo hace. Para cuantificar este ritmo se analiza la dispersión del estimador. Cada simulación individual tiene una varianza σ^2 y, al promediar N simulaciones independientes, dicha varianza se reduce a σ^2/N. Su raíz cuadrada, la desviación estándar, representa el error típico esperado de la estimación, $\sigma/\sqrt{N}$. Para los métodos analizados, σ puede calcularse exactamente:

$$\sigma_\pi = \sqrt{(\pi(4-\pi))} \approx 1{,}642 \qquad \sigma_e = \sqrt{(e(3-e))} \approx 0{,}875 \tag{5}$$

Por lo tanto, ambos métodos convergen con una tasa $O(1/\sqrt{N})$. Esta convergencia es lenta: para reducir el error a la mitad es necesario cuadruplicar N, y para ganar un dígito decimal de precisión, es decir, dividir el error por 10, es necesario multiplicar N por 100. Además, por el Teorema Central del Límite, el error de una estimación sigue aproximadamente una distribución normal, lo que implica que alrededor del 95 % de las ejecuciones tendrá un error inferior a $2\sigma/\sqrt{N}$. Por otro lado, para un mismo N, la aproximación de e resulta casi el doble de precisa que la de π ($\sigma_e/\sigma_\pi \approx 0{,}53$), aunque cada una de sus simulaciones consume más números aleatorios.

Esta lentitud de la convergencia constituye la principal motivación para recurrir a la paralelización: cada dígito adicional multiplica por 100 el trabajo necesario. Existe, entonces, un compromiso entre precisión y costo computacional, ya que aumentar N produce estimaciones estadísticamente más estables, pero también aumenta la cantidad de trabajo que debe realizar el sistema.

3.4.3. Métricas de error

Para cada ejecución se calcula el error absoluto, E_{abs} = |valor estimado − valor real|, utilizando como referencia π y e con 20 dígitos significativos. Para resumir las ejecuciones con un mismo N se utiliza el error RMS (raíz del error cuadrático medio): se elevan al cuadrado los errores, se promedian y se toma la raíz del resultado. Se elige esta medida porque puede compararse directamente con el error teórico $\sigma/\sqrt{N}$. Por último, para comparar ejecuciones con distintos N se utiliza el error normalizado, que divide el error de cada ejecución por el error teórico de su N; si el método se comporta según la teoría, su valor RMS debería ser cercano a 1.

3.5. Limitaciones del método y estrategias de resolución

El Método de Monte Carlo presenta limitaciones propias, independientes de la implementación:

- **Convergencia lenta.** Como el error disminuye con $1/\sqrt{N}$, obtener d decimales correctos requiere del orden de 10^{2d} simulaciones. Por eso Monte Carlo no es un método práctico para calcular muchas cifras de π o e: los métodos deterministas basados en series convergen mucho más rápido, y la elección de Monte Carlo en este proyecto responde a su valor como problema de estudio para la computación paralela (sección 1).
- **Resultado probabilístico.** Cada estimación es aleatoria y su error solo puede acotarse en términos estadísticos (por ejemplo, con un 95 % de confianza), no con una cota determinista.
- **Dependencia del generador de números aleatorios.** Las computadoras generan números pseudoaleatorios, producidos por un algoritmo determinista con un período finito. La validez del método depende de su calidad y, en paralelo, de que las secuencias de los distintos procesos sean independientes.
- **Costo computacional.** Para un mismo nivel de precisión, la cantidad de trabajo crece rápidamente, por lo que el tiempo de ejecución se convierte en el factor limitante.

Las estrategias para enfrentar estas limitaciones actúan sobre distintos aspectos. La **paralelización**, eje de este trabajo, no mejora la tasa de convergencia, pero reduce el tiempo necesario para realizar las N simulaciones. Las técnicas de **reducción de varianza**, como las variables antitéticas o el muestreo estratificado, disminuyen σ y, con ello, el error para un mismo N [2]; los métodos **cuasi-Monte Carlo**, que reemplazan los números pseudoaleatorios por secuencias de baja discrepancia, pueden mejorar la tasa de convergencia. Por último, el uso de **generadores con flujos independientes** para cada proceso asegura la validez estadística de la versión paralela. Algunas de estas estrategias se retoman en la sección 8.

3.6. Métricas de rendimiento paralelo

El rendimiento de la versión paralela se evalúa mediante el *speedup* y la eficiencia [4], [5]:

$$S_P = \frac{T_s}{T_P} \qquad E_P = \frac{S_P}{P} \tag{6}$$

donde T_s es el tiempo de un programa serial independiente, sin MPI, y T_P el de la versión paralela con P procesos. El caso ideal es $S_P = P$ ($E_P = 1$). La Ley de Amdahl [4] acota el *speedup* cuando solo una fracción f del tiempo serial es paralelizable:

$$S_P \leq \frac{1}{(1 - f) + f / P} \tag{7}$$

En este problema, la parte no paralelizable se reduce a combinar los resultados parciales, cuyo costo no depende de N, por lo que f se acerca a 1 a medida que N crece y se espera un *speedup* cercano al ideal para N grandes. En cambio, el costo de la comunicación crece con P, un efecto que se vuelve visible cuando el trabajo por proceso es pequeño. Los experimentos miden escalabilidad fuerte: N fijo y P variable.

4. Implementación

4.1. Versión serial

El núcleo de cada estimación se escribió una sola vez, en los archivos de cabecera `montecarlo_pi.h` y `montecarlo_e.h`, que incluyen tanto la versión serial como la paralela; así, ambas ejecutan exactamente el mismo código de cálculo y solo difieren en cómo se reparte el trabajo (Listado 1).

Listado 1. Núcleo de cálculo de ambas estimaciones (`montecarlo_pi.h` y `montecarlo_e.h`; se omiten los comentarios).

```c
static long long contar_aciertos_pi(long long n)
{
    long long aciertos = 0;

    for (long long i = 0; i < n; i++) {
        double x = drand48();
        double y = drand48();
        if (x * x + y * y <= 1.0) {
            aciertos++;
        }
    }
    return aciertos;
}
```

```c
static long long contar_sumandos_e(long long n)
{
    long long sumandos = 0;

    for (long long i = 0; i < n; i++) {
        double suma = 0.0;
        while (suma <= 1.0) {
            suma += drand48();
            sumandos++;
        }
    }
    return sumandos;
}
```

Cada programa recibe N y, opcionalmente, una semilla (por defecto, 12345). Los contadores son de tipo `long long` (64 bits), ya que en e, con $N = 10^9$, el total de números utilizados alcanza $\approx 2{,}7 \cdot 10^9$ y supera el máximo de un entero de 32 bits. Los números aleatorios se generan con `drand48()`, que devuelve valores uniformes en [0, 1) a partir de un generador congruencial lineal de 48 bits [6]; se prefirió frente a `rand()`, cuyo rango y calidad dependen de la implementación. El generador se inicializa con `srand48(semilla)`: usar una semilla explícita, en lugar de la hora del sistema, hace que cada experimento sea reproducible.

El tiempo se mide únicamente sobre el núcleo de cálculo con `clock_gettime(CLOCK_MONOTONIC)`, un reloj de alta resolución que no se ve afectado por ajustes de la hora del sistema. Cada ejecución imprime una línea CSV con la constante, N, P, la semilla, la estimación, el error absoluto y el tiempo. Los programas se compilan con `make`, que usa `gcc` y `mpicc` con las mismas opciones (`-O2 -Wall -Wextra`), de modo que la diferencia de rendimiento no se deba al compilador.

4.2. Estrategia de paralelización

En ambos programas prácticamente todo el tiempo se concentra en el bucle de simulaciones, cuyo costo crece linealmente con N; el resto tiene un costo constante y despreciable. Por eso se paraleliza ese bucle mediante una

descomposición de dominio sobre las simulaciones, siguiendo el modelo SPMD (*Single Program, Multiple Data*): todos los procesos ejecutan el mismo programa sobre una parte distinta de las simulaciones, calculan un resultado parcial (aciertos en π, números utilizados en e) y al final esos resultados se suman (Figura 2).

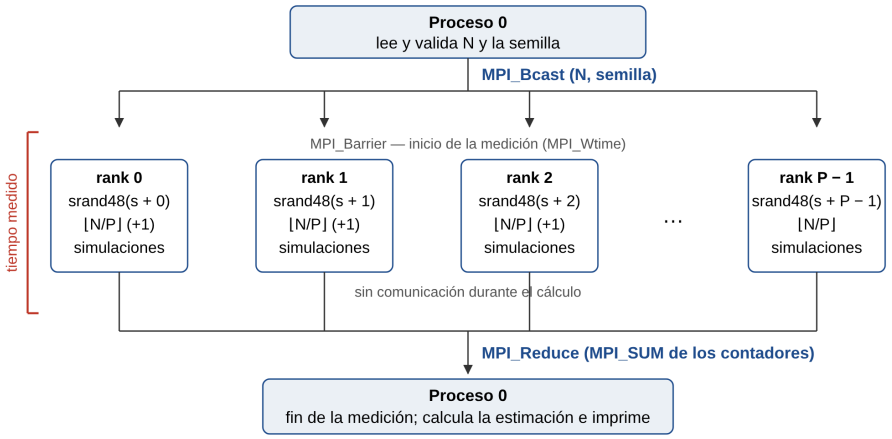


Figura 2. Esquema de la versión paralela: distribución de los parámetros, cálculo local independiente en cada proceso y reducción de los contadores en el proceso 0. La llave roja indica el intervalo de tiempo medido.

Distribución del trabajo. Cada proceso realiza $\lfloor N/P \rfloor$ simulaciones y los primeros $N \bmod P$ procesos realizan una más, de modo que se reparten todas aunque N no sea múltiplo de P, con una diferencia máxima de una simulación entre procesos. Se eligió un reparto estático en lugar de uno dinámico: en π todas las simulaciones cuestan lo mismo, y en e, aunque el costo de cada simulación varía, cada proceso realiza millones de ellas y su costo total se concentra alrededor del promedio (con $N = 10^9$ y $P = 48$, la variación relativa entre procesos es del orden de 0,007 %). Un reparto dinámico solo agregaría comunicación. Además, como distribuir las simulaciones no cambia la cantidad total de muestras, la precisión depende de N y no de P: aumentar N mejora la precisión y aumentar P reduce el tiempo.

4.3. Números aleatorios y comunicación

Para que las simulaciones de distintos procesos sean independientes, cada proceso inicializa su generador con `srand48(semilla + rank)`, donde `rank` es su identificador. El esquema es simple y produce secuencias distintas, pero no garantiza formalmente que sean independientes: todos los procesos recorren el mismo ciclo del generador, de período 2^{48}, desde puntos de partida que no se eligen para evitar solapamientos [7]. Con los tamaños utilizados (como máximo $\approx 2{,}7 \cdot 10^9$ números por proceso) no se espera que esto afecte los resultados, y la sección 6.1 muestra que el error no aumenta con P; su solución se propone en la sección 8.

La comunicación se limita al inicio y al final del cálculo. El proceso 0 lee y valida los argumentos (si son inválidos, finaliza todos los procesos con `MPI_Abort`) y los envía al resto con una única llamada a `MPI_Bcast`. Durante las simulaciones no hay comunicación. Al final, cada proceso aporta su contador parcial a un `MPI_Reduce` con `MPI_SUM` hacia el proceso 0, que calcula la estimación. Se reducen los contadores enteros y no las estimaciones parciales, por lo que la suma es exacta y no es necesario ponderar los procesos con una simulación más. El volumen comunicado no depende de N, y el costo de la reducción crece aproximadamente con $\log_2 P$ [5], [8]. El Listado 2 muestra la parte central de la versión paralela.

Listado 2. Extracto de la parte central de la versión paralela de π (`montecarlo_pi_mpi.c`); la de e es idéntica salvo por la función de cálculo.

```
MPI_Bcast(datos, 2, MPI_LONG_LONG, 0, MPI_COMM_WORLD);   // N y semilla

long long n_local = n / procesos;                         // reparto en bloques
if (rank < n % procesos) {
    n_local++;
}
srand48(semilla + rank);                                  // secuencia propia

MPI_Barrier(MPI_COMM_WORLD);
double inicio = MPI_Wtime();
long long aciertos_local = contar_aciertos_pi(n_local);
long long aciertos_total = 0;
MPI_Reduce(&aciertos_local, &aciertos_total, 1, MPI_LONG_LONG,
           MPI_SUM, 0, MPI_COMM_WORLD);
double fin = MPI_Wtime();
```

4.4. Medición del tiempo

En la versión paralela, los procesos se sincronizan con `MPI_Barrier` y el tiempo se mide con `MPI_Wtime()` desde ese punto hasta después de `MPI_Reduce`. El intervalo medido incluye, por lo tanto, el cálculo y la comunicación necesaria para combinar los resultados, que es el costo propio de la paralelización. Como el proceso 0 recibe la reducción solo cuando todos aportaron su resultado, el tiempo informado corresponde además al proceso más lento.

Quedan fuera de la medición el lanzamiento de los procesos por `mpirun`, la inicialización de MPI y la distribución inicial de los parámetros con `MPI_Bcast`. Las dos primeras son tareas del entorno de ejecución y no del algoritmo, y la versión serial tampoco incluye su arranque ni la lectura de argumentos; la tercera es un único mensaje de 16 bytes, de costo despreciable y constante. De este modo, ambas versiones miden el mismo trabajo y el *speedup* refleja el efecto de la paralelización. La contracara es que el tiempo total de una ejecución completa es mayor que el medido (sección 6.5).

5. Diseño experimental

Los experimentos se realizaron en dos sistemas con características muy distintas (Tabla 1). La información de hardware se obtuvo de los registros generados automáticamente en cada corrida (`lscpu`, `nproc` y la cuota de CPU del contenedor) y de las especificaciones del fabricante.

Tabla 1. Características de los sistemas utilizados.

	Computadora personal (PC)	Nodo de clúster
Procesador	Intel Core i5-10210U (1,6 GHz base, hasta 4,2 GHz con Turbo Boost; 15 W)	2 × AMD EPYC 7B12 (64 núcleos cada uno; hasta 3,3 GHz)
Núcleos físicos / lógicos	4 / 8 (Hyper-Threading)	128 / 128 (sin multihilo simultáneo)
Entorno	Linux sobre WSL 2 (máquina virtual en Windows)	Contenedor Linux con cuota de CPU (48 de los 128 núcleos)
Opciones de `mpirun`	`--oversubscribe`	`--bind-to none --oversubscribe`
Valores de *P*	1, 2, 3, 4, 6, 8	1, 2, 4, 8, 12, 16, 24, 32, 48

En la PC se probaron hasta 8 procesos para observar el efecto de superar los 4 núcleos físicos; en el clúster, el máximo fue la cuota de 48 CPU. En ambos sistemas se usaron $N \in \{10^5, \ldots, 10^9\}$: el valor menor permite observar los costos fijos de la paralelización, y el mayor produce tiempos seriales de unos 12 s (π) y 34 s (e). Cada combinación de constante, N y P, y también la versión serial, se ejecutó 10 veces con semillas distintas, lo que suma 700 ejecuciones en la PC y 1.000 en el clúster.

Las ejecuciones se automatizaron con el script `benchmark.sh`, que compila, registra el sistema y guarda una fila CSV por ejecución. Las configuraciones se ejecutan intercaladas (en cada repetición, la serial y luego la paralela con todos los P), de modo que un cambio en las condiciones del sistema afecte a todas por igual. El script `graficos.py` calcula las métricas y genera los gráficos. Para los tiempos se usa la mediana de las 10 ejecuciones, robusta frente a ejecuciones demoradas por causas externas, y el *speedup* se calcula como mediana(T_s)/mediana(T_P). Para el error se usan todas las ejecuciones con un mismo N; las que repiten semilla y P en ambos sistemas producen la misma estimación y se cuentan una sola vez, por lo que quedan 1.200 ejecuciones distintas.

6. Resultados y análisis

6.1. Precisión: convergencia y error

La Figura 3 muestra las estimaciones obtenidas en todas las ejecuciones en función de N: en ambas constantes, la dispersión alrededor del valor real se reduce a medida que N aumenta, como predice la Ley de los Grandes Números.

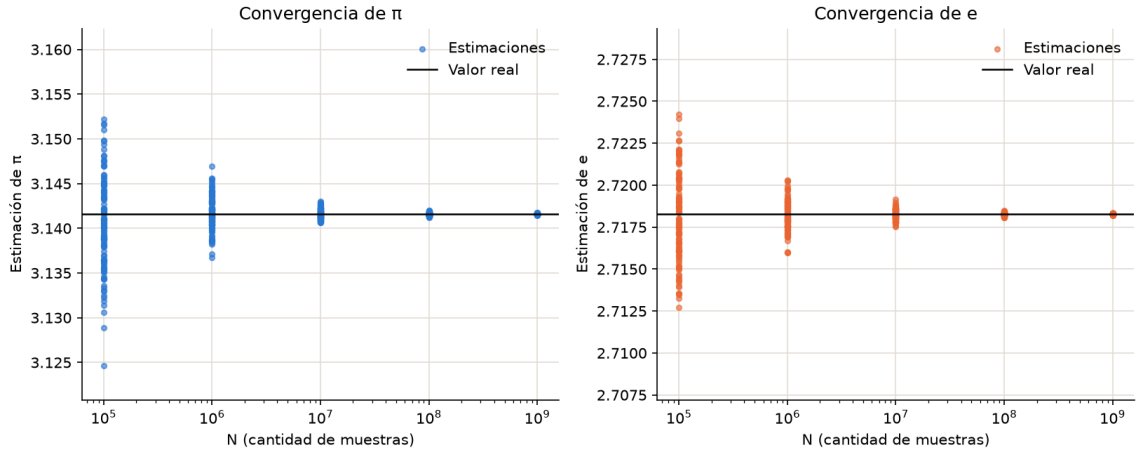


Figura 3. Estimaciones de π y e de todas las ejecuciones en función de N (escala logarítmica en N). La línea negra indica el valor real.

La Figura 4 (izquierda) muestra el error RMS en función de N en escala logarítmica, donde una relación del tipo error $\propto N^{-1/2}$ aparece como una recta de pendiente −0,5. Las pendientes ajustadas son −0,505 para π y −0,497 para e, prácticamente iguales a la teórica, y la Tabla 2 muestra que los valores medidos difieren de los teóricos en menos de un 12 % en todos los casos y en menos de un 4 % para $N \geq 10^8$. También se verifica la aproximación normal: el 95,2 % de las ejecuciones de π y el 95,7 % de las de e tienen un error menor que $2\sigma/\sqrt{N}$, frente al 95,4 % esperado.

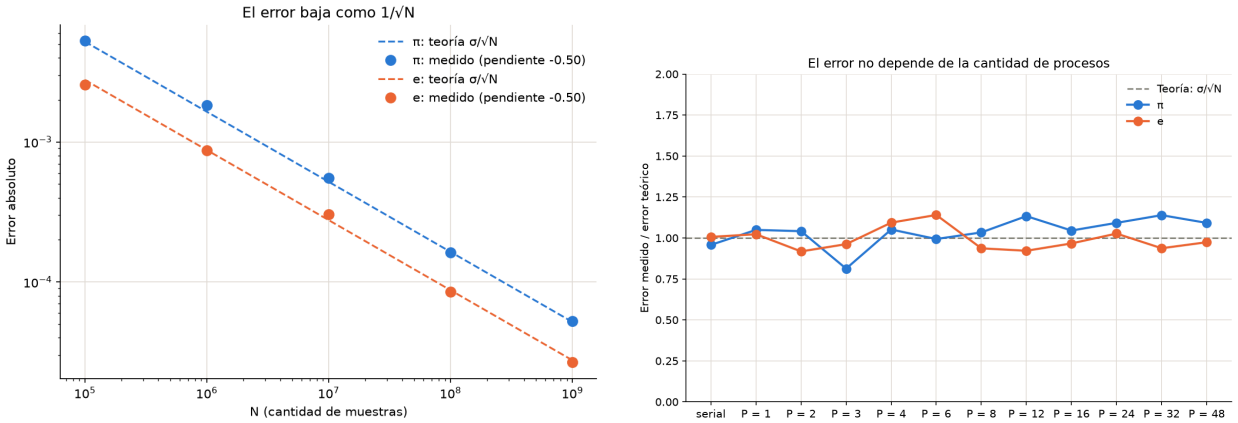


Figura 4. Izquierda: error RMS en función de N (escala log-log), frente a la recta teórica $\sigma/\sqrt{N}$, con la pendiente ajustada. Derecha: error RMS normalizado por $\sigma/\sqrt{N}$ para la versión serial y cada cantidad de procesos; el valor 1 corresponde a la predicción teórica.

Tabla 2. Error RMS medido y error típico teórico $\sigma/\sqrt{N}$ (120 ejecuciones distintas por constante y valor de N).

N	π medido	π teórico	e medido	e teórico
10^5	$5{,}25 \cdot 10^{-3}$	$5{,}19 \cdot 10^{-3}$	$2{,}57 \cdot 10^{-3}$	$2{,}77 \cdot 10^{-3}$
10^6	$1{,}82 \cdot 10^{-3}$	$1{,}64 \cdot 10^{-3}$	$8{,}73 \cdot 10^{-4}$	$8{,}75 \cdot 10^{-4}$
10^7	$5{,}55 \cdot 10^{-4}$	$5{,}19 \cdot 10^{-4}$	$3{,}04 \cdot 10^{-4}$	$2{,}77 \cdot 10^{-4}$
10^8	$1{,}62 \cdot 10^{-4}$	$1{,}64 \cdot 10^{-4}$	$8{,}52 \cdot 10^{-5}$	$8{,}75 \cdot 10^{-5}$
10^9	$5{,}29 \cdot 10^{-5}$	$5{,}19 \cdot 10^{-5}$	$2{,}68 \cdot 10^{-5}$	$2{,}77 \cdot 10^{-5}$

Con $N = 10^9$, el error típico es de aproximadamente $5 \cdot 10^{-5}$ para π y $3 \cdot 10^{-5}$ para e, es decir, cuatro decimales correctos; ganar uno más requeriría $N = 10^{11}$, unos 20 minutos de ejecución serial para π. Por último, la Figura 4 (derecha) muestra que el error normalizado se mantiene entre 0,81 y 1,14 para todas las cantidades de procesos, sin tendencia con P; como cada valor resume 50 ejecuciones, su incertidumbre es de aproximadamente 10 %, por lo que las diferencias son compatibles con la fluctuación estadística. Paralelizar, entonces, no degrada la precisión, y no se observan indicios de correlación entre las secuencias aleatorias de los procesos, aunque esto no alcanza para demostrar su independencia.

6.2. Tiempos de ejecución

La Figura 5 muestra el tiempo mediano en función de N en el clúster. En escala logarítmica, todas las curvas, incluida la serial, son rectas paralelas de pendiente 1: en todas las versiones el trabajo es proporcional a N, por lo que el tiempo crece linealmente con N. Repartir ese trabajo entre P procesos divide el tiempo por un factor constante, que en escala logarítmica equivale a desplazar la recta hacia abajo sin cambiar su pendiente; cada duplicación de P lo reduce en un factor cercano a 2. Solo para N pequeños y muchos procesos las curvas se aplanan, porque los costos fijos de sincronización y comunicación se vuelven comparables con el cálculo.

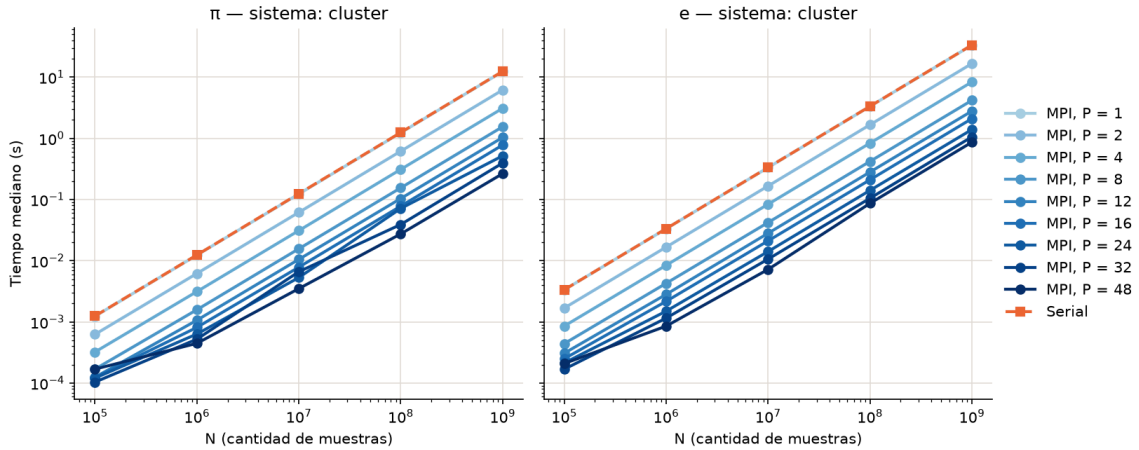


Figura 5. Tiempo mediano en función de N en el clúster, para la versión serial y para cada cantidad de procesos (escala log-log).

El costo por simulación es prácticamente igual en ambos sistemas: con $N = 10^9$, la versión serial tarda 12,4 s (clúster) y 12,1 s (PC) para π, unos 12 ns por punto, y 33,4 s y 34,1 s para e, unos 34 ns por simulación. Cada núcleo rinde, por lo tanto, lo mismo en los dos sistemas, y las diferencias que se analizan a continuación se deben a cómo escalan con P. Por número aleatorio generado, π cuesta unos 6 ns y e unos 12 ns: el bucle interno de e tiene una cantidad aleatoria de iteraciones, lo que impide al procesador anticipar cuándo termina, y cada suma depende de la anterior (sección 8). Por último, en el clúster la versión MPI con un solo proceso tarda lo mismo que la serial (diferencias menores al 1,2 %): MPI no agrega un costo apreciable al cálculo.

6.3. Speedup y eficiencia en el clúster

La Figura 6 muestra el *speedup* en el clúster para cada N, y la Tabla 3 resume los valores con $N = 10^9$ en ambos sistemas.

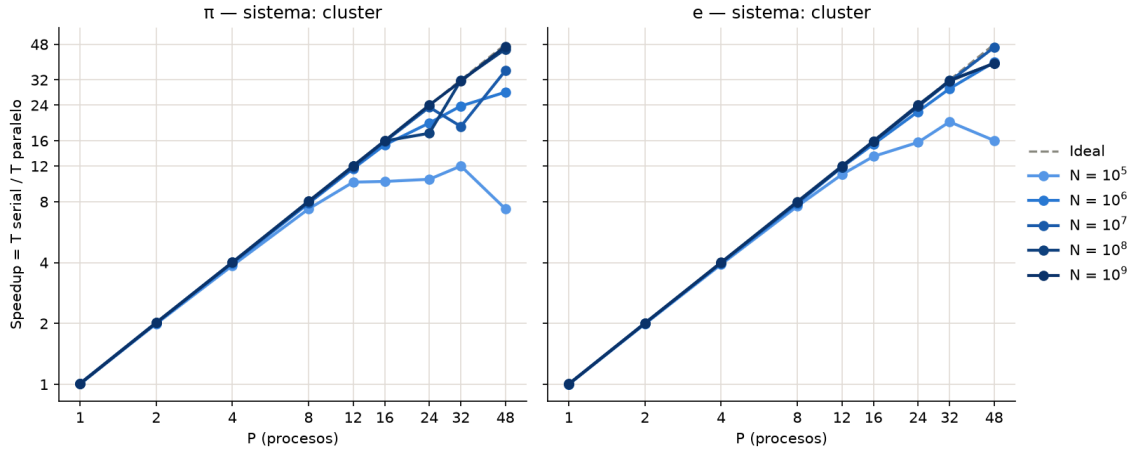


Figura 6. *Speedup* en el clúster en función de P para cada N (escala log-log). La línea punteada indica el *speedup* ideal.

Tabla 3. *Speedup* (S) y eficiencia (E) con $N = 10^9$ (mediana de 10 ejecuciones).

P	Clúster — π		Clúster — e		PC — π		PC — e	
	S	E	S	E	S	E	S	E
1	1,01	1,01	1,00	1,00	1,00	1,00	1,00	1,00
2	2,02	1,01	2,01	1,00	1,47	0,74	1,85	0,93
3	—	—	—	—	1,89	0,63	2,57	0,86
4	4,02	1,00	4,01	1,00	2,09	0,52	3,28	0,82
6	—	—	—	—	2,46	0,41	4,35	0,72
8	8,03	1,00	8,01	1,00	2,60	0,33	4,17	0,52
12	12,02	1,00	11,99	1,00	—	—	—	—
16	16,00	1,00	15,94	1,00	—	—	—	—
24	24,01	1,00	24,03	1,00	—	—	—	—
32	31,50	0,98	31,64	0,99	—	—	—	—
48	46,92	0,98	38,68	0,81	—	—	—	—

***N* grande.** Para $N \geq 10^8$ el *speedup* es prácticamente ideal hasta 32 procesos en ambas constantes, con eficiencias de 0,98 o superiores, y π alcanza 46,9 con 48 procesos. Esto confirma lo esperado del diseño: los procesos no se comunican durante el cálculo y la única reducción final es despreciable frente a segundos de cómputo.

Variabilidad. Algunos valores no siguen la tendencia: e con 48 procesos (eficiencia 0,81) y los puntos aislados de la Figura 6 (π con $N = 10^8$ y $P = 24$, y con $N = 10^7$ y $P = 32$). En esas configuraciones, las 10 ejecuciones se dividen en dos grupos, uno con *speedup* prácticamente ideal (por ejemplo, 47,8 para e con 48 procesos) y otro más lento, y la mediana queda en el grupo lento o entre ambos. La diferencia no proviene del código: todas las ejecuciones realizan la misma cantidad de trabajo, el retardo aparece incluso con un único proceso y su magnitud es siempre similar (≈ 1,75 veces en π y ≈ 1,25 en e). Como `MPI_Reduce` espera a todos los procesos, basta con que uno sea más lento para demorar la ejecución completa, por lo que el efecto es más frecuente cuantos más procesos se utilizan. Su causa no pudo determinarse; entre las posibles se encuentran la competencia con otras tareas que comparten la máquina física o diferencias de rendimiento entre núcleos. En las ejecuciones sin este efecto, la variación entre repeticiones es del orden de 0,3 % o menor.

***N* pequeño.** Con $N = 10^5$, el *speedup* de π se estanca en 10 a 12 y cae a 7,4 con 48 procesos: cada proceso realiza unas 2.000 simulaciones, decenas de microsegundos de cálculo, comparables con el costo de sincronizar y reducir. Es la manifestación práctica de la Ley de Amdahl, agravada porque la parte no paralelizable crece con *P*. La estimación de e escala mejor con el mismo *N* (19,9 con 32 procesos) porque cada simulación cuesta 2,7 veces más y hay más cálculo por cada comunicación. Para superar una eficiencia de 0,9 con 48 procesos fueron necesarias del orden de $2 \cdot 10^6$ simulaciones por proceso en π y $2 \cdot 10^5$ en e.

6.4. Speedup en la PC y comparación entre sistemas

En la PC los resultados son muy distintos (Tabla 3 y Figura 7): con $N = 10^9$, el *speedup* máximo es 2,60 para π y 4,35 para e, y la eficiencia es menor que 1 aun con 2 a 4 procesos, sin superar los núcleos físicos, en especial en π (0,74 con 2 procesos). Como el mismo programa escala de forma casi ideal en el clúster, la causa está en la plataforma:

- **Frecuencia variable.** El i5-10210U es un procesador de notebook de 15 W: con un núcleo activo trabaja cerca de su frecuencia máxima, pero con varios activos el límite de potencia y temperatura la reduce. El tiempo serial se mide a mayor frecuencia que el paralelo, lo que reduce el *speedup*.
- **Hyper-Threading.** Con más de 4 procesos, dos procesos comparten un núcleo físico. e todavía mejora de 4 a 6 procesos (de 3,28 a 4,35), posiblemente porque su bucle tiene más tiempos muertos que el otro hilo puede aprovechar, mientras que π mejora poco (de 2,09 a 2,46). Con 8 procesos, e empeora, posiblemente porque los procesos compiten con los del propio sistema.

- **Virtualización.** WSL 2 es una máquina virtual de Windows: el sistema anfitrión decide cómo asignar los núcleos y agrega variabilidad (hasta 12 % entre repeticiones).

Con las mediciones disponibles no es posible cuantificar cuánto aporta cada factor, para lo cual habría que registrar la frecuencia del procesador. Además, para $N \leq 10^6$ se obtienen eficiencias mayores que 1 (hasta 3,1), que no corresponden a un *speedup* superlineal real: son tiempos de milisegundos en los que la versión serial, la primera de cada repetición, resulta más lenta que la MPI con un proceso, posiblemente porque el procesador aún no alcanzó su frecuencia de trabajo. Por eso, en la PC solo se consideran interpretables los resultados con $N \geq 10^7$.

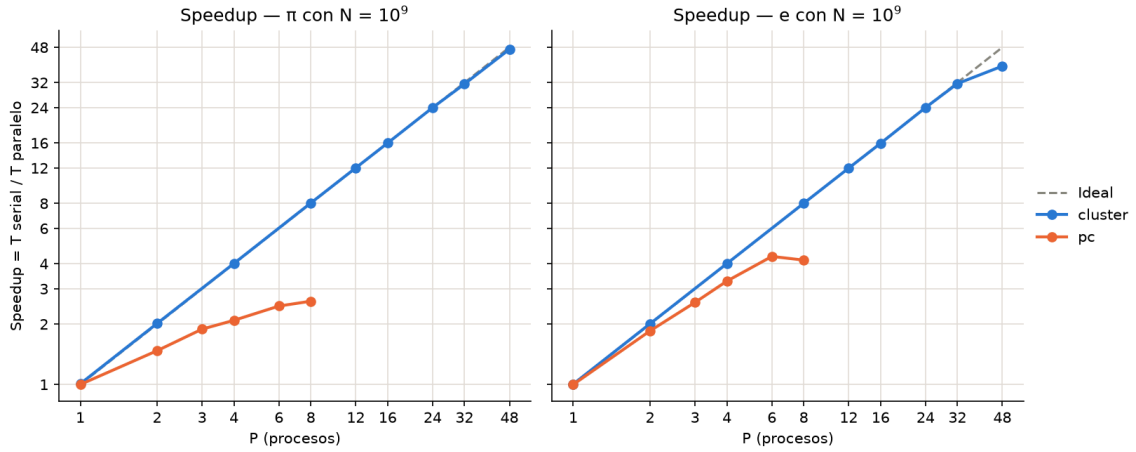


Figura 7. Comparación del *speedup* de ambos sistemas con $N = 10^9$ (escala log-log). La línea punteada indica el *speedup* ideal.

En tiempo absoluto, estimar π con $N = 10^9$ lleva unos 12 s en serie, 4,7 s en la PC con 8 procesos y 0,26 s en el clúster con 48 procesos, 47 veces menos que la versión serial del mismo clúster; para e, los tiempos son unos 34 s, 7,8 s (PC, 6 procesos) y 0,86 s (clúster; 0,70 s en las ejecuciones más rápidas). El *speedup* depende, entonces, tanto de la plataforma como del algoritmo: un problema trivialmente paralelo escala de forma casi ideal cuando cada proceso dispone de un núcleo dedicado y de frecuencia estable, mientras que en una notebook la frecuencia variable, los núcleos lógicos y la virtualización limitan el beneficio mucho antes de agotar el paralelismo del problema.

6.5. Limitaciones del estudio

- El tiempo medido excluye el lanzamiento de los procesos y la inicialización de MPI; para N pequeños, el tiempo total de una ejecución completa estaría dominado por esos costos.
- Las mediciones en el clúster se realizaron en un único nodo, sin comunicación entre nodos a través de la red.
- Con 10 repeticiones, la mediana deja de ser robusta cuando la mitad o más de las ejecuciones resultan más lentas.
- No se registró la frecuencia del procesador de la PC ni el estado de la máquina del clúster durante las mediciones, por lo que las causas de la menor escalabilidad en la PC se plantean como hipótesis y la de las ejecuciones más lentas en el clúster no pudo determinarse.

7. Conclusiones

Se implementó la estimación de π y e mediante el Método de Monte Carlo en C, con una versión serial y una paralela con MPI, y se evaluó su precisión y rendimiento en una computadora personal y en un nodo de clúster.

En cuanto a la precisión, los resultados coinciden con la teoría: el error disminuye como $1/\sqrt{N}$ (pendientes de −0,505 y −0,497), su magnitud coincide con $\sigma/\sqrt{N}$ y cerca del 95 % de las ejecuciones queda dentro de dos errores típicos. Con $N = 10^9$ se obtienen cuatro decimales correctos de cada constante. La convergencia lenta es

la principal limitación del método, ya que cada decimal adicional requiere multiplicar por 100 la cantidad de simulaciones, y es lo que justifica la paralelización.

La independencia entre simulaciones permitió paralelizar con un reparto estático y una única reducción final, sin comunicación durante el cálculo, y se verificó que repartir el trabajo no modifica la precisión. En el clúster, el *speedup* fue prácticamente ideal para N grandes, con eficiencias de 0,98 o superiores hasta 32 procesos y hasta 48 en π; las caídas observadas se deben a ejecuciones puntualmente más lentas, cuya causa no pudo determinarse pero que no provienen del algoritmo. Para N pequeños, el costo de sincronización y comunicación limita el *speedup*, de acuerdo con la Ley de Amdahl.

En la PC, el *speedup* fue mucho menor (2,6 para π y 4,4 para e) a pesar de que cada núcleo rinde lo mismo que en el clúster: la frecuencia variable, el uso de núcleos lógicos y la virtualización limitan el beneficio. El rendimiento paralelo depende tanto del algoritmo como de la plataforma. Por último, la mediana de múltiples ejecuciones resultó adecuada para obtener resultados estables, aunque en un entorno compartido conviene complementarla con más repeticiones o con el tiempo mínimo.

8. Mejoras futuras

Como la paralelización ya escala de forma casi ideal, el margen de mejora está en el costo de cada simulación, en la independencia de los números aleatorios entre procesos y en la información que reporta el programa. Las mejoras 8.1 a 8.4 se evaluaron en un banco de pruebas aparte (versión serial en una notebook, compilada con `-O2`) sin integrarse al código, por lo que sus tiempos son preliminares y sirven para comparar las variantes entre sí (Tabla 4).

8.1. Generador xoshiro256 con salto por proceso.** Reemplazar `drand48()` por xoshiro256** [9], una función que el compilador puede insertar dentro del bucle y cuyo estado permanece en registros. Su función de salto avanza la secuencia 2^{128} posiciones: si el proceso de rango r la aplica r veces, cada proceso recorre un tramo distinto y la ausencia de solapamiento queda garantizada por construcción [10], lo que resuelve la limitación de la sección 4.3.

8.2. Un número aleatorio por punto en π. Un único número de 64 bits alcanza para las dos coordenadas (31 bits cada una), lo que reduce a la mitad las llamadas al generador. El sesgo introducido es del orden de 10^{-9}, muy inferior al error estadístico.

8.3. Aritmética entera. Evaluar la condición de π y acumular la suma de e con enteros en lugar de `double`. En π la ganancia es pequeña; en e es importante, porque cada suma depende de la anterior y la suma entera es más rápida y exacta.

8.4. Bucle sin saltos impredecibles en e. El bucle interno `while (suma <= 1.0)` termina tras una cantidad aleatoria de iteraciones, lo que hace fallar la predicción de saltos del procesador en casi cada simulación y es la causa más probable de que cada número aleatorio cueste el doble en e que en π (sección 6.2). Reemplazar los dos bucles por uno solo, que reinicia la suma con una operación aritmética en lugar de una condición, mantiene el mismo estimador y es la mejora de mayor impacto.

Tabla 4. Efecto acumulado de las mejoras en el banco de pruebas (versión serial, ns por iteración; menos es mejor).

Paso	π	e
Código actual (`drand48`)	5,4	18,6
+ xoshiro256** (8.1)	3,3	14,6
+ un número por punto (8.2)	2,5	—
+ aritmética entera (8.3)	2,3	9,8
+ bucle plano con enteros (8.3 y 8.4)	—	6,2
Mejora total	**≈ 2,3×**	**≈ 3×**

Estas mejoras aceleran por igual la versión serial y la paralela, por lo que no deberían modificar el *speedup* relativo sino el tiempo absoluto; para una comparación honesta, T_s debería medirse con la versión serial mejorada, y harían falta N mayores para mantener la eficiencia, ya que los costos fijos de comunicación pesarían más.

8.5. Error estándar en la salida. Informar en cada ejecución el error estándar estimado a partir de las propias muestras (en π, $4\sqrt{(\hat{p}(1-\hat{p})/N}$, con $\hat{p}$ = aciertos/N; en e, a partir de la varianza muestral de K), de modo que cada ejecución informe su intervalo de confianza sin necesidad de conocer el valor real.

8.6. Procedimiento experimental. Aumentar las repeticiones o informar también el tiempo mínimo, para detectar distribuciones bimodales como las del clúster; fijar cada proceso a un núcleo y registrar en qué núcleo se ejecuta cada uno, para identificar el origen de las ejecuciones más lentas; registrar la frecuencia del procesador de la PC; y medir también el tiempo de la ejecución completa.

8.7. Otras alternativas. El modelo híbrido MPI + OpenMP no aportaría ganancia dentro de un único nodo y solo tendría sentido al escalar a varios nodos. Las técnicas de reducción de varianza (variables antitéticas, cuasi-Monte Carlo) mejoran la precisión para un mismo N, pero modifican el método, por lo que corresponderían a una variante a comparar. La vectorización SIMD y el uso de GPU son posibles, pero requieren reescribir el núcleo de cálculo.

9. Bibliografía

En la redacción y revisión de este informe se utilizó como herramienta de apoyo el asistente de inteligencia artificial Claude [11]; el diseño, la implementación, los experimentos y el análisis son responsabilidad de los autores.

[1] N. Metropolis y S. Ulam, “The Monte Carlo method,” *Journal of the American Statistical Association*, vol. 44, n.º 247, pp. 335–341, 1949.

[2] C. P. Robert y G. Casella, *Monte Carlo Statistical Methods*, 2.ª ed. Nueva York: Springer, 2004.

[3] K. G. Russell, “Estimating the value of e by simulation,” *The American Statistician*, vol. 45, n.º 1, pp. 66–68, 1991.

[4] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” en *Proc. AFIPS Spring Joint Computer Conference*, 1967, pp. 483–485.

[5] P. S. Pacheco, *An Introduction to Parallel Programming*. Burlington, MA: Morgan Kaufmann, 2011.

[6] The Open Group, “drand48 — generate uniformly distributed pseudo-random numbers,” *The Open Group Base Specifications*, Issue 7 (IEEE Std 1003.1-2017).

[7] Linux man-pages project, “drand48(3) — Linux manual page.” [En línea]. Disponible: https://man7.org/linux/man-pages/man3/drand48.3.html

[8] Message Passing Interface Forum, *MPI: A Message-Passing Interface Standard*, versión 4.0, 2021.

[9] D. Blackman y S. Vigna, “Scrambled linear pseudorandom number generators,” *ACM Transactions on Mathematical Software*, vol. 47, n.º 4, art. 36, 2021.

[10] P. L’Ecuyer, D. Munger, B. Oreshkin y R. Simard, “Random numbers for parallel computers: Requirements and methods, with emphasis on GPUs,” *Mathematics and Computers in Simulation*, vol. 135, pp. 3–17, 2017.

[11] Anthropic, “Claude” (asistente de inteligencia artificial basado en modelos de lenguaje), 2026. [En línea]. Disponible: https://claude.ai. Utilizado como herramienta de apoyo en la redacción y revisión del informe.